Pip Architecture — F2 + F3: persist structured context, then event-driven recompute

FOLIOS · FOLIOSHQ.COM

Source: `docs/audit-2026-06-17.md` → X6 Pip Architecture Review (the "FORM" sequence), and CLAUDE.md item 48 lever (3)/(4). F1 (one shared `buildAccountContext()`) shipped June 19; it cleared the path for this. F3 is the audit's named **70–90% Pip-cost cut**.

This plan covers F2 (the foundation) and F3 (the cut). Build F2 first, then F3 on top. Work on the branch; do NOT deploy — Chris fast-forwards `main` after he tests (F1 playbook).

THE WASTE CLASS (verified file:line — this is what F3 kills)

`api/pip-state-refresh.js` issues a Haiku call **per account** to produce a 2–3 sentence `state_prose` + a JSON sidecar, upserted to `folio_pip_account_state`. It is fired by **timers**, not events:

1. `src/App.jsx:335` ("Part 9") — every 6h, top-10 accounts by `last_interaction_at`.
2. `src/views/pip/PipView.jsx:170` — on chat open, every 6h, up to 20 "stale" accounts.
3. `src/views/accounts/AccountDetail.jsx:154` — the manual "resync Pip memory" button (fine).

There **is** already a partial event-gate: `findStaleAccountIds` / Part 9 only refresh an account when `last_interaction_at > generated_at`. But that signal **only bumps when a meeting is logged** — closing a task, editing a task,

adding an account update, or a project status change never advances `last_interaction_at`, so:

- those changes **never trigger a recompute** (a real staleness leak), AND
- the 6h timers still re-evaluate the top-N every window (the waste item 48 names).

What the Haiku output is actually used for (so we cut the right thing)

- `state_prose` → consumed **only** by chat's compact render path (`pipContext.renderAccountCached`, used when `cachedState` is fresh and not brief mode). It is a token-saving substitute for the full per-account context in chat.
- `health_signal` / `momentum` / `risk_flags` (the sidecar) → **no consumer anywhere in `src/`** (verified by grep). Dead data. We don't remove the columns (additive-only schema rule) but we stop pretending they matter; the fingerprint/gate ignore them.
- The **row** is load-bearing for OTHER writers: `operator_*` (operator-run) and `lessons_learned` (the compression pass) are consumed by the UI. F3 touches only the `pip-state-refresh`-owned fields; the operator/compression fields are untouched.

> *Note for a future session (NOT this pass): because `state_prose`'s only live consumer is chat's compact view, and the full per-account context is now deterministic + free (F1), there is a **bigger** cut available — retire the Haiku `state_prose` entirely and let chat read the deterministic `renderAccountContext`. That is a **behavior change to what reaches the chat model**, so it needs Chris's explicit call after he's tested F3. F3 keeps the Haiku output byte-for-byte identical (same `buildPrompt`); it only changes **when** it fires.*

F2 — persist the structured context (the foundation)

Add to `folio_pip_account_state` (additive, `if not exists`; apply via Supabase MCP + fold into `schema.sql`):

column	type	purpose
`context_struct`	`jsonb`	the `buildAccountContext()` structured output (`{sections:[…]}`) — the durable record of "what Pip knew", and the substrate for F6 (pgvector) + operator delta later. Write-only for now (no consumer changes → zero behavior risk).
`context_fingerprint`	`text`	deterministic hash over the **stable** signal inputs (ids/dates/counts — never relative-time). The content-gate F3 reads.
`context_checked_at`	`timestamptz`	last time the server evaluated this account for recompute (set on every pass, even when the Haiku call is skipped). Lets the client apply a short cooldown without a global timer.

Pure helper in `src/lib/accountContext.js` (same pure module — no supabase/React/fetch):

```
export function computeContextFingerprint(account) -> string
```

- Builds a deterministic, **order-independent** object from the merged bundle and hashes it with a tiny dependency-free FNV-1a string hash (this is change-detection, not security).
- **STABLE inputs only** (the Sanity-Pass crux — see Risk Controls):
  - account: `id, name, status, status_override, tier, account_type, owner_user_id, last_interaction_at, hash(objective), join(systems)`
  - meetings: `count + max(updated_at) + max(meeting_date)` (catches add/remove + edit/summarize)

    - tasks: `openCount + doneCount + max(updated_at)` (close bumps `updated_at` via the DB trigger AND flips `done`; add/remove changes count)
    - contacts: `count + hash(join(name|relationship_role|is_primary))`
    - projects: `count + join(status) + max(status_update.at)`
    - updates: `count + max(update_date)`

- **NEVER** includes `Date.now()`, "Xd ago", `[overdue Nd]`, or any value derived from the current time. A unit test mocks "one day later" over identical data and asserts the fingerprint is **unchanged** — that test is the drift lock for this property.

Writer: `api/pip-state-refresh.js`. It already loads meetings/tasks/contacts/projects per account; we add one cheap `folio_account_updates` query (so `context_struct` is complete). On each pass it maps the loaded rows into the `buildAccountContext` bundle shape, computes `context_struct = buildAccountContext(bundle, {surface:"chat"})` and `context_fingerprint = computeContextFingerprint(bundle)`, and writes both + `context_checked_at`. (The **Haiku prompt** still uses the existing `buildPrompt` — we do NOT reroute the model input through `buildAccountContext`; that would change model behavior. F2's structured context is for persistence + the gate only.)

F3 — event-driven recompute (the two-tier gate)

The whole design avoids the trap that would **invert** the goal: if the client and server computed the fingerprint independently and diverged by one field, fingerprints would never match → recompute every time → *more* calls. So the fingerprint is computed

server-side only; the client uses a cheap **recency** gate, never a hash. They cannot drift.

Tier 1 — client recency gate (cheap, broad; replaces the timers)

Move the single broad sweep into `src/App.jsx` Part 9 (it already loads `accounts`, global `meetings`, and global `allItems` /tasks via realtime — the signals we need):

- Per account, compute `lastSignalTime = max( account.last_interaction_at, max(meeting.updated_at | meeting.created_at for this account), max(task.updated_at for this account) )`.
- Candidate iff **no state row** OR `lastSignalTime > state.generated_at`.
- **Remove the 6h timer.** The effect runs when data is ready (deps: userId, accounts, meetings, allItems, states). Guard against realtime thrash with:
  - an in-flight `Set` ref of account ids currently being refreshed (don't re-POST), and
  - a per-account "already POSTed at this lastSignalTime" ref so a realtime tick that doesn't advance `lastSignalTime` doesn't re-POST.
  - (After a successful refresh `generated_at` advances on refetch, so the gate closes naturally — the refs only cover the in-flight window.)
- Cap the batch (20). Major-tier first when the cap bites.
- **Remove PipView's separate on-open sweep** — App's broad sweep supersedes it (App Coherence Rule: one sweep, not two). Chat still reads the freshest `cachedState`.

This widens the trigger set from meeting-only to **meeting + task** changes (and, via Tier 2, anything else), and makes it fire on the event (data-ready after a realtime change) instead of on a clock.

Tier 2 — server content gate (precise; the cost guarantee)

In `api/pip-state-refresh.js`, for each requested account, **before** the Haiku call:

1. Rebuild the bundle from the freshly-loaded rows.
2. `fp = computeContextFingerprint(bundle)`.
3. Read the stored `context_fingerprint`.
4. If `fp === stored` **and not** `force` → **skip the Haiku call.** Write only `context_checked_at = now()` (and refresh `context_struct` if absent). Count it `skipped`.
5. Else → do the Haiku call as today, then write `state_prose` + sidecar + `context_struct`
   - `context_fingerprint = fp` + `generated_at` + `stale_at` + `context_checked_at`.

Return `{ refreshed, skipped }` for observability. The manual "resync" button passes `force:true` (an explicit user action always recomputes).

Why both tiers: Tier 1 alone over-fires on no-op/irrelevant writes (it's recency, not content). Tier 2 alone would require the client to POST every account every run (the server would still load+hash everything). Tier 1 cheaply narrows to "something happened"; Tier 2 confirms "something that *matters* happened" and is the hard $0-guarantee. This mirrors the codebase's documented belt-and-suspenders pattern (the two SW update paths).

COST MATH

- **Today:** up to ~10 calls/6h (App Part 9) + up to ~20/6h (PipView), bounded loosely by the meeting-only recency gate — but it re-evaluates top-N every 6h

window and misses task-only changes. Order of tens of Haiku calls/day on a normal day, most of them no-ops.

- **After F3:** a Haiku call fires **only when an account's content fingerprint actually changed** since its last compute. On a typical day Chris meaningfully touches a handful of accounts → a handful of recomputes; everything else is a $0 Tier-1 skip (never POSTed) or a $0 Tier-2 skip (POSTed but fingerprint-identical). Estimated **70–90% reduction** on the `pip-state-refresh` line, matching item 48, with **zero freshness loss** (and actually *better* freshness — task/update changes now trigger a refresh that the old gate missed).

RISK CONTROLS (Sanity-Pass Rule — this is a silent-failure surface)

1. **The fingerprint must be time-stable.** The single worst failure: a relative-time value sneaks into the fingerprint → it changes daily → recompute daily → savings evaporate silently. Mitigation: stable inputs only (stored timestamps/ids/counts), and a unit test that mocks `Date.now()` +1 day over identical data and asserts the fingerprint is identical. If that test ever fails, the gate is leaking.
2. **No client/server fingerprint divergence.** Computed server-side only; client uses recency. They cannot disagree.
3. **Fail toward freshness, never staleness.** Tier 1 candidate logic and Tier 2 errs on the side of recomputing (no row, or any newer signal) — a missed skip costs a few cents; a missed *recompute* would show stale state to the user. We never skip when in doubt (unknown fingerprint, parse trouble, `force`).
4. **No behavior change to what reaches the chat model.** The Haiku `buildPrompt` input is unchanged → `state_prose` is identical in content; only its recompute timing changes. `context_struct` is write-only (no consumer) → zero risk. Chat reads the same `cachedState`, just fresher on real change.
5. **Realtime thrash guard.** In-flight ref + per-account last-POSTed-signal ref prevent the App sweep from re-POSTing on every realtime tick.

6. **Gate every commit:** `npx vite build && npx vitest run && node scripts/check-guards.js && node scripts/test-api-imports.js`. 323-test baseline stays green; new tests add to it.
7. **DB:** additive columns only (`if not exists`), applied via Supabase MCP + folded into `schema.sql`. No destructive change.

BUILD ORDER (one concern per commit, gated between each)

1. **F2-a — schema + helper + test.** Add the three columns (MCP migration + `schema.sql`); add `computeContextFingerprint` to `accountContext.js` ; add fingerprint tests (incl. the time-stability lock). No behavior change yet.
2. **F2-b — server persists.** `pip-state-refresh` loads updates, computes + writes `context_struct` / `context_fingerprint` / `context_checked_at` on every pass (still always does the Haiku call at this step — persistence only). Verify rows populate.
3. **F3-a — server Tier-2 skip.** `pip-state-refresh` skips the Haiku call when the fingerprint is unchanged (honor `force`); returns `{refreshed, skipped}` . The biggest $ win lands here and is safe (skips only true no-ops).
4. **F3-b — client Tier-1 rewrite.** App Part 9 broad recency sweep (no timer); remove PipView's separate sweep; manual button passes `force:true` . Widens signals + drops timers.
5. **Docs.** `docs/upgrades.md` entry; square the "nightly cron" wording while here if cheap; regenerate PDFs.

OUT OF SCOPE (named so they're not silently dropped)

- Retiring the Haiku `state_prose` entirely (chat → deterministic context). Bigger cut, behavior change — Chris's call after testing F3.
- F5 (agent loop), F6 (pgvector recall) — later sessions; `context_struct` is F6's substrate.
- The compression pass (App Part 10) is already content-gated (5+ new corrections); left as-is.